

Principal Component Analysis

PCA (Principal Component Analysis) is a dimensionality reduction technique used to reduce the number of features while retaining most of the information in the dataset.

Principal Components

Principle components are

PC1 (First Principle Component)

- Captures the maximum variance in the data.
- Most Important Component

PC2 (Second Principle Component)

- Captures the remaining variance
- Perpendicular to PC1

PC3, PC4,

- Capture progressively less variance

Original Features

Height'

Explained Variance Ratio

Explained Variance Ratio tells us how much information each principal component retains.

Example

Component	Variance Explained
PC1	

Interpretation

- PC1 Contains 70% of the information
- PC2 Contains 20% of the information
- PC1+PC2 Contain 90% of the information

So we keep

Scree Plot

Purpose of Scree Plot

- Find how many components to keep
- Identify the "elbow point"
- Components after the elbow contribute very little information.

Example

If: |

2. Highly Correlated Features

Example:

Height (cm) Height (inch)

4.

PCA Workflow

Data -> Standardization -> PCA -> Principal Components -> Train Model

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import pandas as pd
```

Load dataset

```
iris = load_iris()
X = iris.data
```

Standardize data

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Apply PCA

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

Explained Variance Ratio

```
print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
```

PCA DataFrame

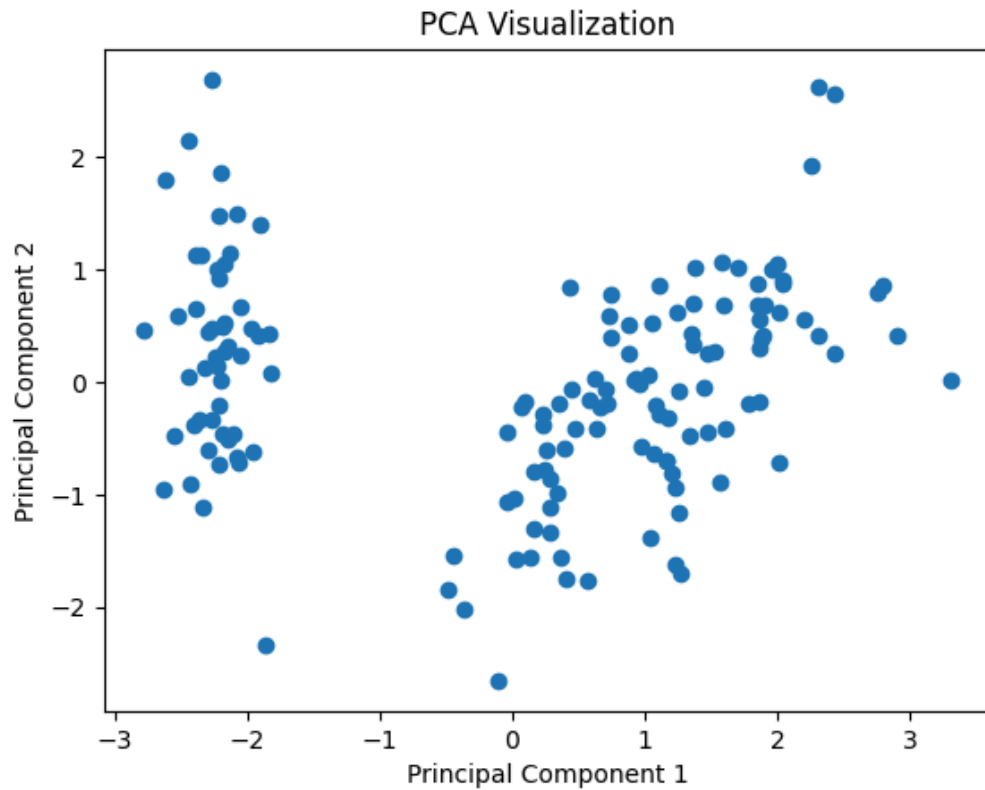
```
df = pd.DataFrame(
    X_pca,
    columns=["PC1", "PC2"]
)
print(df.head())
```

Scatter Plot

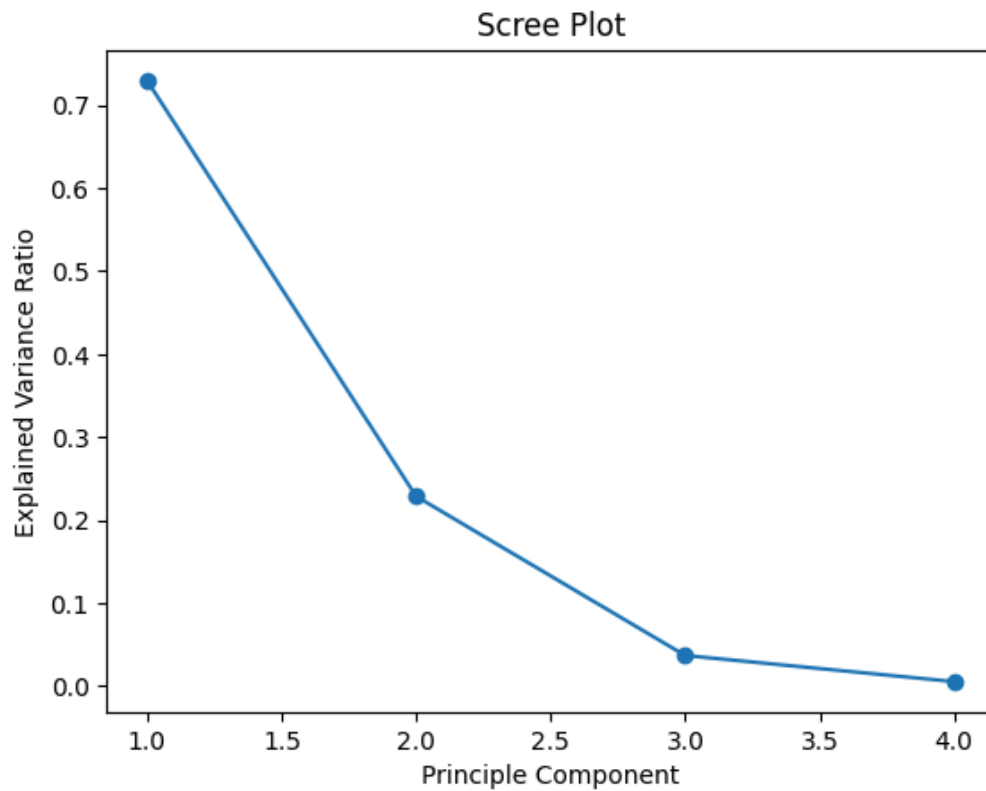
```
plt.scatter(X_pca[:,0], X_pca[:,1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualization")
plt.show()
```

Explained Variance Ratio:
[0.72962445 0.22850762]

	PC1	PC2
0	-2.264703	0.480027
1	-2.080961	-0.674134
2	-2.364229	-0.341908
3	-2.299384	-0.597395
4	-2.389842	0.646835



```
# Scree Plot
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
pca=PCA()
pca.fit(X_scaled)
plt.plot(
    range(1,len(pca.explained_variance_ratio_)+1),
    pca.explained_variance_ratio_,
    marker='o'
)
plt.xlabel("Principle Component")
plt.ylabel("Explained Variance Ratio")
plt.title("Scree Plot")
plt.show()
```



Cross Validation

Cross Validation is a Technique used to evaluate a machine learning model by training and testing it on different subsets of the data.

Its Main Purpose is to check how well the model generalizes to unseen data.

Why is a Single

K-Fold Cross Validation

In K-Fold Cross Validation, The dataset is divided into K-Equal Parts.

Example

Iteration 1: Test -> Fold 1 Train -> Fold 2 + Fold 3 + Fold 4 + Fold 5

Iteration 2: (92 + 90 + 94 + 91 + 90)

K-Fold vs Stratified K-Fold

K-Fold	Stratified K-Fold
Random Splits into K Folds	Preserves Class distribution
Suitable for balanced data	suitable for imbalanced data
May Create uneven classed	Keeps class
General Purpose	Mostly used in

#K-Fold Cluster

```

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import KFold, cross_val_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Model
model = DecisionTreeClassifier()

# K-Fold
kf = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
scores = cross_val_score(
    model,
    X,
    y,
    cv=kf
)
print("Scores:", scores)
print("Mean Accuracy:", scores.mean())

Scores: [1.         1.         0.93333333 0.93333333 0.93333333]
Mean Accuracy: 0.9600000000000002

#stratiedKFold

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Model
model = DecisionTreeClassifier()

# Stratified K-Fold
skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

scores = cross_val_score(
    model,
    X,
    y,
    cv=skf
)

```

```
print("Scores:", scores)
print("Mean Accuracy:", scores.mean())
```

```
Scores: [1.          0.96666667 0.93333333 0.96666667 0.9        ]
Mean Accuracy: 0.9533333333333335
```

Hyperparameter Tuning

Hyperparameter Tuning is the process of finding the best hyperparameter values for a machine learning model to improve performance.

What are Hyperparameters?

Hyperparameter are settings that are chosen before training the model.

Examples

Algorithm	Hyperparameters
KNN	n_neighbors(K)
Decision Tree	max_depth,min_samples_split
Random Forest	n_estimators,min_samples_split

For KNN:

K=3 K=5 K=7

Goal:

Find the best combination of hyperparameters that gives the highest model performance.

Why Hyperparameter Tuning:

Different Hyperparameter values can produce different results.

K=1 -> Accuracy = 85% K=3 -> Accuracy = 90% K=5 -> Accuracy = 95%

The model performance changes when the hyperparameter changes.

Hyperparameter Tuning Methods

1.GridSearchCV

GridSearchCV tries all possible combinations of hyperparameters and selects the combination with the best performance.

Example

Suppose:

max_depth=[3,5,7] min_samples_split=[2,4]

Comparison

Method	Search Type	Speed
Grid Search CV	Tests all Combinations	Slow
RandomizedSearchCV	Tests Random Combinations	Faster
Optuna	Learns and chooses better combinations	Faster

GridSearchCV Code

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
```

Dataset

```
iris=load_iris()
X=iris.data
y=iris.target
```

```
model=DecisionTreeClassifier()
```

```
param_grid={
    'max_depth':[2,3,4,5],
    'min_samples_split':[2,4,6]}
```

#Grid Search

```
grid=GridSearchCV(
    model,
    param_grid,
    cv=5,
    scoring='accuracy'
)
```

```
grid.fit(X,y)
print("Best Parameters:",grid.best_params_)
print("Best Score:",grid.best_score_)
```

```
Best Parameters: {'max_depth': 3, 'min_samples_split': 4}
Best Score: 0.9733333333333334
```

RandomizedSearchCV Code

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV
```

Dataset

```
iris=load_iris()
X=iris.data
y=iris.target
```

```
model=DecisionTreeClassifier()
```

```
param_dist={
    'max_depth':[2,3,4,5,6,7,8],
    'min_samples_split':[2,4,6,8,10]}
```

```
random_search=RandomizedSearchCV(model,param_distributions=param_dist,n_iter=6,cv=5,random_sta
```

```
random_search.fit(X,y)
```


Successfully installed colorlog-6.10.1 optuna-4.9.0

```
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

# Load Dataset
iris = load_iris()
X = iris.data
y = iris.target

# Objective Function
def objective(trial):
    max_depth = trial.suggest_int("max_depth", 2, 10)
    min_samples_split = trial.suggest_int("min_samples_split", 2, 10)

    model = DecisionTreeClassifier(
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        random_state=42
    )

    score = cross_val_score(model, X, y, cv=5).mean()
    return score

# Create and Optimize Study
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=20)

# Results
print("Best Parameters:")
print(study.best_params)

print("\nBest Score:")
print(study.best_value)

Best Parameters:
{'max_depth': 3, 'min_samples_split': 9}

Best Score:
0.9733333333333334
```